ls – list all the files and folders in a directory

-a	List all content including hidden folders	-d	List current directory
-l	Include file metadata	-S	Sort by size (largest first) (-r for reverse)
-h	Human-readable file size	-t	Sort by modification timestamp (recent first)

pwd – prints the current directory

cd – change the directory

../	Goes up 1 directory	~	Goes back to home directory

touch – creates an empty file

rm – delete a file

rmdir – deletes directories

-i	Ask before delete	-r	Deletes recursively for directories

mkdir – creates empty directory

cp – copy file from source to destination

mv – move file from source to destination

-i	Ask before overwrite if file exists	-r	Copies recursively, including folders
-n	Do not overwrite		

---

man [command] – shows the manual page of a command

f or spacebar	Scrolls page down	q	Quits the manual
b	Scrolls page up	-k	Shows man pages containing the command (what?)
/word	Searches for the word in the man page	-f	Shows man pages with a short description that match the command (whatis?)
n	Finds next match for word search	-S[section #]	Open man page in specific number

head – show first 10 lines
tail – show last 10 lines

-n [number]	Specifies number of lines to output

grep [pattern] [filename]

-n	Show list of line numbers that contains the pattern	–color	Highlights instances of he match
-c	Only shows the number of lines that matched	-i	Ignore case sensitivity

sort [filename]

-r	Reverse sort	-k [number]	Sort by specified field number -k 2 – sort by 2nd word
-t [char]	Set delimiting character sort -t ‘,’ -k 2 file.csv	-n	Numerical sort

wc [filename] – outputs the number of lines, words and bytes of a file

Pipe | – used to send output of a command as input to another

Ex. ls -l | head

Redirection

<	Data from a file is sent to a command
>	Data from a command is sent to a file, will overwrite
>>	Similar to >, but will append output

Vi text editor

i	Insert at cursor	dd	Delete line
a	Insert after cursor	dw	Delete word
A	Insert at end of line	yy	Put line into clipboard
ESC	Exits insert mode	P	Paste text
u	Undo last change	:w	Save file but keep open
U	Undo all changes to line	:q!	Quit file without saving
dd	Delete line	:wq	Save the file and quit

Nano text editor

Ctrl-k	Deletes a line and copies to clipboard	Ctrl-_	Goes to the line number inputted
Ctrl-u	Pastes the copied line	Alt-U	undo
Ctrl-o	Save without exit	Alt-E	redo
Ctrl-x	Save with exit		

---

du – presents the disk usage in directories

df – show the disk statistics of the file system. Lists the capacity, used, available space, % used

-h	Human-readable size format

-d 1	Lists the parent directory only

ln [target file] [link name] – to create hard link or soft link

-s	To create a soft link

chmod vyz [filename] – used to change permissions of file

Special Characters

() – to group commands together	(pwd; ls –l) > out.txt
(()) – for arithmetic operations	echo $((1+1))
{} – to delimit variable names	echo ${var1}_hello
\ – to print special characters	echo \$2

Control Statements

; – run commands consecutively and independently	Date ; ls -l
&& – only perform command 2 if command 1 is successful	rm file1 && echo ‘Success!’
|| – perform command 2 if command 1 is not successful (vice versa)	rm file 1 || ‘Delete failed’

Comparators – use ‘test’ command or [ ] (space is important)

File	-e (exists)
String	=, !=, /<, />
Arithmetic	-eq, –le, -lt, -ge, -gt
Logic	-a, -o

ps – lists the processes “process status”

kill [pid] – terminates a process

Alias – personal short keyword for a command Ex. alias ll=“ls -l””

set – to set shell characteristics

-o	Turns on
+o	Turns off
noclobber	Prevent overwriting files by redirection
verbose	Print shell input lines as tead

#!/bin/sh – to spawn a ne BASH shell

./[script path/name] – to execute a script

Script Code

CONDITIONALS – IF/ELSE

```
if command / condition
then
     do something
Elif command / condition
then
     do something else
else
     do something else
fi
```

Example:

```
read var1
if [ $var1 -eq 0 ]
then
     echo “Zero”
Elif [ $var1 -gt 1]
then
     echo “Positive”
else
     echo “Negative”
fi
```

CONDITIONALS - CASE

```
case "variable" in
value1)
     do something
     ;;
value2)
     do something else
     ;;
*)
     do default action
esac
```

Example:

```
echo “Delete file1? (Y/N)”
read var1
case “$var1” in
Y|y)
      rm file1
      echo “file1 deleted.”
      ;;
N|n)
      echo “Did not delete.”
      ;;
*)
      echo “Invalid input.”
esac
```

LOOPS - WHILE

```
while condition
do
   do something
done
```

Example :

```
i=1
while [ i lt 6 ]
do
     echo “printing $i”
     i=$(($i + 1))
done
```

LOOPS - FOR

```
for var in list
do
  do something
done
```

Example 1:

```
for i in 1 2 3 4 5
do
      echo $i
done
```

Example 2:

```
list=“str1 str2 str3 str4”
for i in $list
do
      echo $i
done
```

```
get_file_name()
```

```
{
  echo -n "Enter filename: "
  read mfile
}

print_no_file()
{
  echo "$mfile: No such file or directory"
}

while [ 1 ]
do
  clear
  echo "File Manager 1.0"
  echo "Please choose an option"
  echo "1 - Move a file to the recycle bin"
  echo "2 - Retrieve a file from the recycle bin"
  echo "3 - Empty the recycle bin"
  echo "4 - Delete a file permanently"
  echo "5 - Exit"
  echo -n "Choose option: "
  read option
  case $option in
    1)
      get_file_name
      if [ -f $mfile ]; then
       mv $mfile $HOME/.junk
      else
       print_no_file
      fi;;
    2)
      get_file_name
      if [ -f $HOME/.junk/$mfile ]; then
       mv $HOME/.junk/$mfile $HOME
      else
       echo "$mfile: No such file or directory"
      fi;;
    3)
      echo -n "Are you sure you want to empty recycle bin? (y/n)"
      read answer
      if [ $answer="y" -o $answer="Y" ]; then
       rm $HOME/.junk/*
      fi;;
    4)
      get_file_name
      if [ -f $mfile ]; then
       rm -i $mfile
      else
```

```
                print_no_file
               fi;;
             5)
               exit;;
             *)
           esac
         done
```

```
  GNU nano 6.2                                    Quizon_filesort.txt *
#!/bin/bash


create_directories()
{


dir_array=("Documents" "Images" "Music" "Others")

current_dir=$(pwd)
echo "The current directory is: $current_dir"

for dir in "${dir_array[@]}"; do
        if [ ! -d "$current_dir/$dir" ]; then
                mkdir "$current_dir/$dir"
                echo "$dir created"
        else
                echo "$dir already exists"
        fi
done
}

echo "CREATING DIRECTORIES"
create_directories

echo "MOVING FILES"
for file in * ; do
        if [ -f "$file" ]; then
                file_name="$file"
                file_extension="${file_name##*.}"
                echo "$file_extension"
                if [[ "$file_extension" == "pdf" || "$file_extension" == "txt"
]]; then
                        echo mv  "$file" "Documents/"
                         mv  "$file" "Documents/"
                elif [[ "$file_extension" == "jpg" || "$file_extension" ==
"png" ]]; then
```

```
                        echo mv "$file" "Images/"
                        mv "$file" "Images/"
                elif [[ "$file_extension" == "mp3" || "$file_extension" ==
"wav" ]]; then
                        echo mv "$file" "Music/"
                        mv "$file" "Music/"
                else
                        echo mv  "$file" "Others/"
                        mv "$file" "Others/"
                fi
        else
                echo "$file is  not a file. No move done"
        fi
done
```